

Methods

Describe and Create a Method in Java

In Java, a method is a block of code that performs a specific task. Methods are used to execute a set of statements, which can be invoked when needed, making the code reusable and modular. A method in Java can perform operations, return a result, and take parameters for customization.

Components of a Method:

1. Method Declaration:

- **Access Modifier** (optional): Defines the visibility of the method (e.g., `public`, `private`, `protected`).
- **Return Type**: Specifies the type of value the method will return (e.g., `int`, `String`, `void`).
- **Method Name**: The name of the method that is used to call it.
- **Parameter List** (optional): A list of input parameters enclosed in parentheses. The parameters define the data that can be passed to the method.
- **Method Body**: A block of code enclosed in curly braces `{}` that defines what the method does.

2. Method Signature: The method name and parameter list together form the method signature. The return type is not part of the signature.

3. Return Statement (optional): A method may return a value, and the `return` statement is used to return that value.

Steps to Create a Method

1. **Define the Method**: Write the method declaration and body with the necessary logic.
 2. **Call the Method**: The method can be called (invoked) from within other methods or constructors.
 3. **Return a Value (if applicable)**: If the method has a return type, you must use the `return` keyword to return the result.
-

Types of Methods:

1. **Void Methods**: These methods do not return any value (i.e., their return type is `void`).

2. **Return Methods:** These methods return a specific value (e.g., `int`, `String`, `boolean`, etc.).
-

Example 1: Void Method (Method with no return value)

In this example, we create a method that displays a message to the console.

```
public class MethodExample {  
  
    // Method that prints a greeting message (void method)  
    public static void greet() {  
        System.out.println("Hello, welcome to Java!");  
    }  
  
    public static void main(String[] args) {  
        // Calling the greet method  
        greet(); // Invoking the method  
    }  
}
```

- **Explanation:**

- The method `greet()` is declared with the return type `void`, meaning it doesn't return any value.
 - It prints the message "Hello, welcome to Java!" when called.
 - The `greet()` method is invoked inside the `main()` method.
-

Example 2: Method with Return Value

This example demonstrates a method that adds two numbers and returns the result.

```
public class MethodExample {  
  
    // Method that returns the sum of two integers  
    public static int add(int a, int b) {  
        return a + b; // Return the sum of a and b  
    }  
  
    public static void main(String[] args) {  
        // Calling the add method and storing the result  
        int sum = add(5, 10);  
    }  
}
```

```
        System.out.println("The sum is: " + sum); // Output: The sum is: 15
    }
}
```

- **Explanation:**

- The method `add(int a, int b)` takes two integer parameters and returns their sum as an integer.
- The `add()` method is called with arguments `5` and `10`, and the returned result is stored in the variable `sum`.
- The `main()` method then prints the result of the sum.

Example 3: Method with Multiple Parameters

Here's an example of a method that takes two strings and concatenates them.

```
public class MethodExample {

    // Method that concatenates two strings and returns the result
    public static String concatenate(String str1, String str2) {
        return str1 + " " + str2; // Concatenating str1 and str2 with a
space
    }

    public static void main(String[] args) {
        // Calling the concatenate method
        String result = concatenate("Hello", "World");
        System.out.println(result); // Output: Hello World
    }
}
```

- **Explanation:**

- The method `concatenate(String str1, String str2)` takes two string parameters and returns a single string that is the concatenation of both strings with a space in between.
- It is called with the values `"Hello"` and `"World"`, and the result is printed.

Summary of Method Syntax

- **Method Declaration:**

```
accessModifier returnType methodName(parameterList) {  
    // method body  
}
```

- `accessModifier`: (e.g., `public`, `private`, etc.)
 - `returnType`: The type of value the method returns (`void` if no value is returned).
 - `methodName`: The name of the method.
 - `parameterList`: Optional list of parameters (comma-separated) passed to the method.
- **Method Call:**
 - For a static method:

```
methodName(arguments);
```

- For an instance method:

```
object.methodName(arguments);
```
